

### **Write code logic for the given Scenarios:**

1. A healthcare startup wants to build a model to predict the onset of diabetes. The dataset has 50 features, but the team wants to use only the most relevant ones. How can they apply feature selection effectively?

#### **Answer:**

This is a classification model.

1. After collecting the raw data and preprocess the data, using the feature selection we choose the most relevant features alone.

2. In feature selection, we choose RFE(Recursive Feature Elimination) to reduce the features to only the most relevant ones because the wrapper methods captures feature dependencies well.

3. We pass the algorithms like LogisticRegression, Random Forest, Decision Tree, Support Vector, Gradient Boosting algorithm into RFE method to obtain the optimized number of features.

4. Then for each different set of reduced number of features are passed in to different classification algorithms to find the best model out of all.

5. Save the best model.

---

2. You're creating a new project for managing student data. The project name is student\_portal. What is the step-by-step command to start it? write logic for this django project

#### **Answer:**

#### **Logic below are for Streamlit Project:**

##### **1. Open a file - app.py and write the below code:**

```
import streamlit as st
import pandas as pd
```

```
#1. Establish database connection
```

```
conn = st.connection(db file name, type="sql", url="db_url")
```

```
#2. Create a database table if it doesn't exist
```

```
with conn.session as session:
```

```
session.execute(""" CREATE TABLE IF NOT EXISTS students ( id INTEGER PRIMARY KEY
AUTOINCREMENT,
name TEXT NOT NULL,
age INTEGER,
grade TEXT,
email TEXT ) """)
session.commit()
```

```
st.title("Student Information System") st.subheader("Add New Student Details")
```

# 3. Create a data entry form

```
with st.form(key="student_form", clear_on_submit=True):
```

```
name = st.text_input("Full Name*", placeholder="xxxx")
age = st.number_input("Age", min_value=5, max_value=100, value=18)
grade = st.selectbox("Grade/Class")
email = st.text_input("Email Address")
```

```
# Required field verification notice
st.caption("* Required field")
```

```
# Form submission button
submit_button = st.form_submit_button(label="Save Student Details")
```

# 4. Handle form submission

```
if submit_button:
    if not name:
        st.error("Submission failed: Please enter a student name.")
    else:
        with conn.session as session:
            query = """ INSERT INTO students (name, age, grade, email) VALUES
(:name, :age, :grade, :email) """
            session.execute(query, {"name": name, "age": age, "grade": grade, "email": email})
            session.commit()
            st.success(f'Successfully saved records for {name}!')
```

## **2.Install Streamlit :**

```
pip install streamlit
```

## **3.To Run the application from terminal:**

```
streamlit run app.py
```

---

3. A company is analyzing customer purchase patterns with 200+ behavioral features. How can they reduce dimensionality without losing predictive power?

### **Answer:**

This is a clustering problem.

Here, we can use dimensionality reduction - UMAP(Uniform manifold approximation and Projection), to reduce dimensionality without losing predictive power. It's advanced geometry to map high-dimensional data into a lower space. This makes it easier for clustering algorithms to find highly accurate boundaries.

---

4. A digital library wants to recommend books to readers based on what similar readers liked. How should they design this system?

**Answer:**

Here, we use User based recommendation system.

1. Collect raw user dataset and raw book dataset
2. Create pivot table using values as Ratings
3. Create Similarity tables between users from `sklearn.metrics.pairwise` using Cosine metric
4. To predict the non filled ratings for non-read books by users, calculate Score value using formula.
5. Receive the user input. Find the similarity between input user and other users by converting `user_similarity` into table
6. Find similar user for input user using cosine table
7. Create a list of `book_id` for similar user and create a list of `book_id` for input user
8. Filter the `book_id` of similar user which is not present in input user `book_id` list
9. Check Filtered `book_id` in prediction table to get the ratings and select only the highest rated `book_id` from the list based on the threshold value( range of rating)
10. With the list of highest rated `book_id`, we can recommended books by getting the book names from the book dataset table

---

5. A bank wants to assess the risk level of credit applicants using only the most important financial indicators. How can they reduce the number of features?

**Answer:**

As it involves assessment on risk level on credit applicants(Finance), to reduce number of features we can use Embedded Methods from Feature Selection method - Tree-Based

Ensembles (like XGBoost or LightGBM) combined with Lasso ( $L_1$ ) Regularization for more accuracy.

#### Step 1: Clean and Prepare the Applicant Data

- Gather all raw data features from the credit applications
- Handle missing values and convert categorical data into numerical values.
- Split dataset into a Training Set to build the models and a test Set to test the accuracy.

#### Step 2: Apply Layer 1 – Lasso ( $L_1$ ) Regularization (Linear Filtering)

- Train a Logistic Regression model with an  $L_1$  penalty on your training data to predict default risk.
- Lasso mathematically penalizes the model for using too many features. It forces the weights (coefficients) of weak, highly correlated, or irrelevant credit features to become exactly zero
- Drop any financial column whose weight hits zero (e.g., if two features track the exact same debt metric, Lasso will keep one and completely delete the other). This aggressively shrinks your feature pool.

#### Step 3: Apply Layer 2 – Tree-Based Ensembles (Non-Linear Interaction)

- Take the remaining, survival features from Step 2 and feed them into a tree-based model like XGBoost or LightGBM.
- Train these trees to split applicants into risk categories based on complex interactions (e.g., a low income might only be dangerous if combined with a high credit utilization).
- Extract the Feature Importance Scores from the trained model. This ranks columns based on how much predictive accuracy or "gain" each feature added across thousands of decision trees

#### Step 4: Define the Final Threshold and Intersect

- Look at the ranked feature importance list from the tree-based model.
- Establish a cutoff threshold (e.g., keep only the top 30 features that contribute to 95% of the model's predictive power) and discard the low-scoring variables.
- The remaining columns represent your optimized feature set. They have passed two distinct tests: they possess linear significance (Lasso) and hold powerful non-linear predictive strength (XGBoost/LightGBM).

#### Step 5: Train and Validate the Final Production Model

- Re-train your final credit risk model using only this highly refined, reduced set of columns.
  - Test the model on your unseen test Set to ensure it predicts high-risk applicants accurately without overfitting.
  - Deploy the streamlined model. It will now run faster, require less data pipeline maintenance, and be easier to document for financial regulatory compliance
- 

6. A news app wants to recommend articles based on both article similarity and user reading history. How can they implement a hybrid system?

**Answer:**

Hybrid recommendation system can be implemented using content based filtering and collaborative filtering .Then combining the scores from both models, rank the final article feed.

**Step 1: Filter Out Read Articles**

- Gather the identification details for every article in the database.
- Check the current user's reading history.
- Remove any articles the user has already read to create a clean list of fresh candidate articles.

**Step 2: Calculate Collaborative Filtering Score**

- Group users who have identical reading patterns to the current user.
- Look at what those similar users are reading right now.
- Assign a Collaborative Score to each candidate article based on how popular it is among that peer group.

**Step 3: Calculate Content-Based Filtering Scores**

- Analyze the text features of the current user's historical articles (topics, categories, text embeddings).
- Analyze the text features of the new candidate articles.
- Run a similarity calculation (cosine similarity).
- Assign a Content Score to each candidate article based on how closely its keywords match the user's favorite topics.

**Step 4: Normalise the Scores**

- Scale all calculated scores to a uniform range between 0.0 and 1.0.This step is mandatory because collaborative scores (e.g., clicks or raw counts) and content scores (e.g., percentages or similarity decimals) use completely different scales. Normalisation prevents one calculation engine from overpowering the other.

**Step 5: Fuse the Scores Using a Weight (Alpha)**

- Define a weight factor called Alpha (a value between 0.0 and 1.0) to balance personalization versus trends.
- Multiply the normalised Collaborative Score by Alpha.
- Multiply the normalised Content Score by (1 minus Alpha).
- Add these two numbers together to generate one definitive Final Hybrid Score for every candidate article.

#### Step 6: Rank and Serve the Feed

- Take the final list of candidate articles and sort them from the highest hybrid score to the lowest hybrid score.
- Serve the top-ranked articles directly to the user's application screen feed.

7. You're building a spam detection model and have thousands of text features from emails. How do you identify the most useful ones?

#### **Answers:**

This is a classification model.

From All Text Columns



Using Variance Threshold-- Removes ultra-rare words



Using Chi-Square Filter -- Drops thousands of columns to ~500



Using TSVD(Truncated Singular Value Decomposition) --: Blends 500 sparse columns into 50 dense concepts



Using Logistic Regression-- Trains on dense concepts (Switch L1 penalty to L2)

8. An ed-tech platform wants to recommend courses based on what similar learners have enrolled in. What steps would you take?

**Answer:**

Here, we use User based recommendation system to get the recommended courses.

1. Collect raw learner dataset and courses dataset
  2. Create pivot table using values as ratings
  3. Create Similarity tables between learners from `sklearn.metrics.pairwise` using Cosine metric
  4. To predict the non filled ratings for non-enrolled courses by learners, calculate Score value using formula.
  5. Receive the learner input. Find the similarity between input learner and other learners by converting learner\_similarity into table
  6. Find similar learners for input learner using cosine table
  7. Create a list of course\_id for similar learner and create a list of course\_id for input learner
  8. Filter the course\_id of similar learner which is not present in input learner course\_id list
  9. Check Filtered course\_id in prediction table to get the ratings and select only the highest rated course\_id from the list based on the threshold value( range of rating)
  10. With the list of highest rated course\_id, we can recommended courses by getting the course names from the courses dataset table
- 

9. You're developing a car price prediction tool. With 100+ features (e.g., brand, mileage, engine type), how do you reduce complexity?

**Answer:**

This is a Regression model.

- Use Correlation Analysis (Pearson or Spearman) for continuous numerical columns (like mileage or engine size) and ANOVA F-test or Mutual Information Regression for categorical columns (like brand or engine type)
- It instantly drops columns that have a zero or near-zero relationship with a car's price. For example, if a column tracks the "factory assembly line shift number," a filter method will instantly show it has zero correlation with price and delete it.
- To Remove Multi-Collinearity, run a Variance Inflation Factor (VIF) check or a correlation matrix across your features. If two features are highly correlated with each other (e.g., correlation > 0.85), drop one of them. This stops your model from getting confused by redundant data.

- Take the remaining clean columns from your filter steps and feed them into a model-based feature selection tool. Train a tree-based ensemble model like XGBoost, LightGBM, or a Random Forest Regressor on the data. Tree models natively calculate Feature Importance Scores.
- Unlike filter methods, trees look at feature interactions. A tree model will recognize and allows to keep only the features that create powerful pricing combinations.  
(SelectFromModel: The automated programmatic tool that reads those scores, sets a cutoff line, and slices your dataset)
- Rank features based on the final scores . Cut off the list once target threshold (e.g., keeping the top 20–30 features that explain 95% of price fluctuations) hits.
- 100+ columns are compressed into a streamlined, highly predictive matrix that trains faster, prevents overfitting, and accurately targets true car value factors.

---

10.

How do you recommend products to new users who haven't interacted with anything yet?

**Answer:**

This can be achieved using Popularity based recommendation system.

1. Collect raw user dataset and product dataset
2. Create product vector
3. Calculate mean on ratings for each product using user dataset
4. Map the calculated means directly to product vector dataset
5. Get the highest rated product\_id list from product vector as Recommended product\_id list
6. Get the corresponding product name from product dataset as Recommended Products for new user